

Project 3 – Multitasking

Learning Objectives

- Learn how to implement basic cooperative multitasking in your OS.

Overview

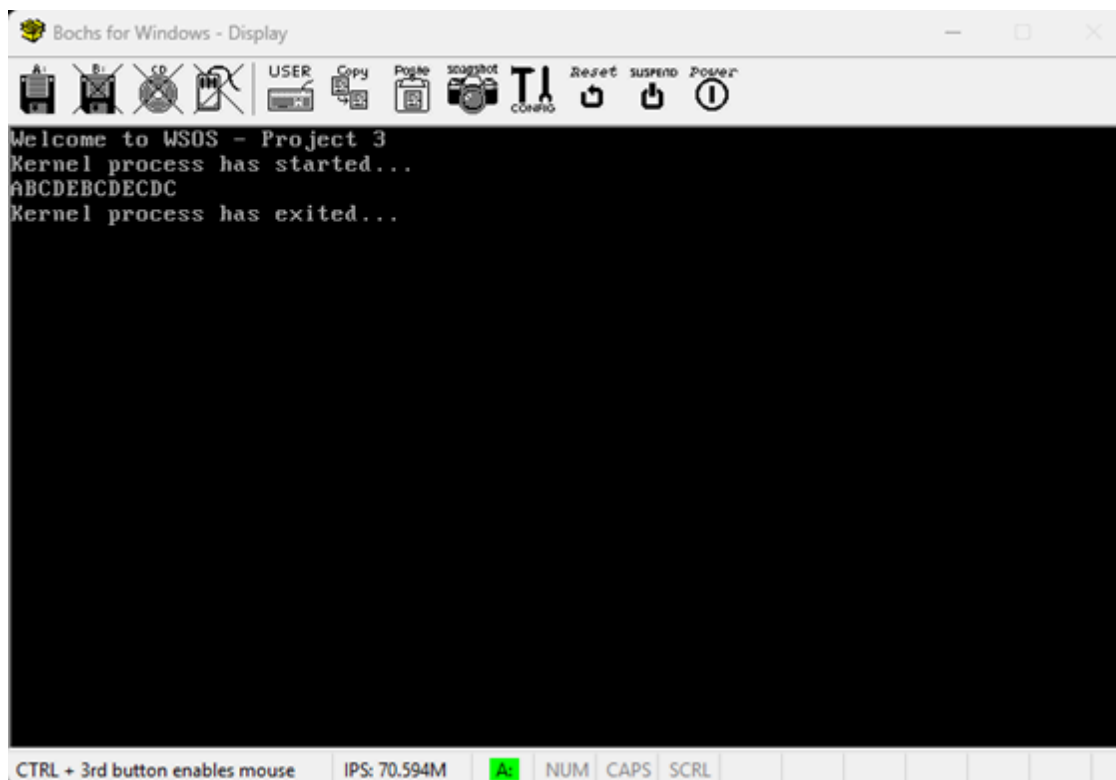
In this project, you will work with user processes, kernel processes, and context switching. Implementing these systems will allow for a very basic cooperative multitasking OS that is able to use a kernel process to schedule a user process to run. Then, the user process can yield back to the kernel process, and repeat. The OS will be capable of context switching between processes to allow for real multitasking.

Multitasking

Although there is included code to perform a context switch, a few more functions must be implemented to allow for true multitasking. These functions that you will implement will allow for the creation of processes, yielding a process, terminating a process, and scheduling the next process. Implement the following functions and modifications:

1. Modify project3.c and project3.h to implement the following functions:
 - a. `int schedule()` determines the next user process to be executed out of all the user processes that are ready to run. To find the next process that is ready to run, iterate through the `proc_t` `processes[MAX_PROCS]` array. Your `schedule` function must implement a round robin scheduling algorithm based on PID. Use the PID of the previously run user process to determine what the next process should be. For example, if the last user process ran has a PID of 3 and PID 5 is the next ready user process, it should be selected to run next. If you iterate to the end of the `processes[]` array, restart at the beginning. To tell the `contextswitch()` function what process to run next, use the address of the element inside of the array to set the `proc_t *nextprocess` pointer.
 - i. returns 1 if a ready user process was found and was selected to run next, else return 0
 - b. `void yield()` allows for processes to free up access to the CPU and allow a new process to execute instead. Yielding is required for cooperative multitasking. When yielding, the currently running process must change its own status to “ready” to show that it will be able to run again sometime in the future. If a user process calls the `yield` function, be sure to track the PID by either setting `proc_t *prevprocess` or storing the PID in an integer, then the kernel must be selected to run next and then call the `contextswitch()` function to context switch to the kernel process. If the kernel process calls the `yield` function, call the `schedule()` function to select the next ready user process, then call the `contextswitch()` function to context switch to the next process.
 - i. returns nothing

- c. void exit() allows a process to terminate itself. When exiting, the currently running process must change its own status to “terminated” to show that it will NOT be able to run again sometime in the future. If a user process terminates itself, this function must select the kernel to run next then context switch to the kernel process. If the kernel process terminates itself, return.
 - i. returns nothing
- d. int createuserprocess(void *func, void *stack) creates a new user process. When the user process is created it must be initialized with a “ready” status, the top of the stack (esp) and base (ebp) both to the stack argument, the instruction to begin executing from (eip) (func), and a process ID. Once initialized it must be added to the process array (proc_t processes[]). This function must not begin executing the process (do not call the function func()), creatproc() only initializes the process and adds it to the process array.
 - i. void *func - the function to execute when our user process is eventually executed
 - ii. void *stack - the address to initialize the process’s stack (esp and ebp) to
 - iii. returns 0 if the process was successfully created and -1 if unsuccessful. Creation can only be unsuccessful if attempting to create more processes than what is defined by the MAX_PROCS macro.
 - iv. Note: the startkernel() function is very similar to this function. Use it as a reference.
- e. int kernel must start the kernel process.
 - i. returns 0
- f. void prockernel() must perform the following actions:
 - i. Creates 5 user processes
 - ii. Prints a start up message
 - iii. Loops, yielding to the next user process
 - iv. When there are no more ready user processes, print a termination message and return
 - v. returns nothing
 - vi. NOTE: The kernel process should run in between each user process yielding so that the kernel process can perform the scheduling. Technically, the order in which processes should execute is:
 - vii. A -> Kernel -> B -> Kernel -> C -> Kernel -> D -> Kernel -> E -> Kernel -> B -> ...
- g. void proca()
 - i. Print “A” then exit.
- h. void procb()
 - i. Print “B”, two times, yielding between prints, then exit.
- i. void procc()
 - i. Print “C”, four times, yielding between prints, then exit.
- j. void procd()
 - i. Print “D”, three times, yielding between prints, then exit.
- k. void proce()
 - i. Print “E”, two times, yielding between prints, then exit.



Output after successfully implementing multitasking

How to Submit

To submit your code, commit your changes to your GitHub Classroom repository and include a link to your repository in the Pilot dropbox.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
Schedule function implemented	1.00
Yield function implemented	1.00
Exit function implemented	1.00
Create process function implemented	1.00
User process functions implemented	1.00
Total	5.00